

Physical AI SDK - Structure & Workflow

Three Paths — Pick the One That Fits Your Workflow

Option 1: Zero Steps

Every example ships with a pre-generated

`METRICS_TABLE.md`

`examples/yolov12/METRICS_TABLE.md`

- CPU, GPU, NPU throughput & latency
- Cross-device comparison table
- Auto-generated from benchmark runs
- **Plan:** expose as a web dashboard

Just open the file — numbers are already there

Option 2: Docker (One Command)

Pre-installed ROCm + RyzenAI environment

```
# Pull & run the container
docker run physical-ai-sdk

# Inside the container:
make benchmark-all-devices metrics
```

- No local installs needed
- Reproducible environment
- Generates `METRICS_TABLE.md` automatically

One command to fresh numbers

Option 3: Manual Setup

Full local installation — 4 steps:

```
# 1. Install ROCm and raizen-ai
./install.sh

# 2. Navigate to example
cd examples/yolov12

# 3. Run benchmarks
make benchmark-all-devices metrics
```

Output: `METRICS_TABLE.md`

Most control, most setup

Easy to Use

Uniform Interface Across Every Example

Make-Based Uniformity

Same commands work in **every** example directory:

```
make benchmark-all-devices metric # all devices
make benchmark-gpu                # GPU only
make benchmark-cpu                # CPU only
make benchmark-npu                # NPU only
make eval                          # evaluation pipeline
```

- Learn once, use everywhere
- No per-model setup surprises
- Consistent output format (`METRICS_TABLE.md`)

Every Example Includes:

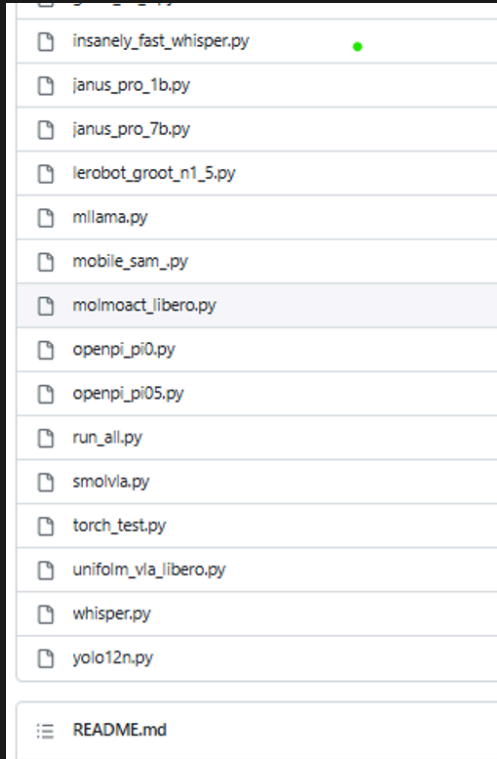
Feature	Description
CPU / GPU / NPU flows	Benchmark each compute device for throughput & latency
Eval pipeline	Sample inputs → sample outputs to visually verify correctness
Guiding documentation	Per-example tutorial to walk you from setup to results

Principle: If you can run one example, you can run them all — same commands, same structure, same outputs.

Contributions — Research Desk → Vertical Software

1. PAVS transforms raw researcher code into production-ready vertical software

What we receive: Research Repo



What we receive: Research Code

```
# rocm-scripts/test/pytorch/yolo12n.py
#.....
# .....
# .....
#.....
# .....
# .....
recalled = gt_labels & detected_classes
recall = len(recalled) / len(gt_labels)
sanity_pass = recall >= RECALL_THRESHOLD
assert sanity_pass, (
    f"recall {recall:.0%} < {RECALL_THRESHOLD:.0%}")

return lats, {
    "architecture": "CNN",
    "task": "object_detection",
    "num_detections": num_detections,
    "recall": round(recall, 4),
}
if __name__ == "__main__":
    sys.exit(run(run_benchmark))
```

Single script, no structure, no device flows

What we deliver: Vertical Software

```
examples/yolov12/
├── Makefile
├── METRICS_TABLE.md
├── README.md
├── app/
│   ├── routes/
│   ├── services/
│   └── main.py & config.py
├── scripts/
│   ├── benchmark.py
│   ├── export_onnx.py
│   ├── profile_model.py
│   ├── run_val.py
│   └── update_metrics_table.py
├── tests/
│   ├── test_api.py
│   ├── test_docker.py
│   ├── test_migraphx.py
│   └── test_yolo_workflow.py
├── weights/
├── datasets/
├── requirements.txt
├── requirements_cpu.txt
└── requirements_npu.txt
```

App, scripts, tests, per-device requirements — production-ready

Flat list of scripts — no structure, no device flows, no Make targets

Physical AI & Vertical Software: Researcher's Desk → Vertical Software — same commands, every device, production-ready.

Contributions(contd.) — Building the AMD AI Ecosystem

2. AIG vs PAVS — Functional Correctness over Operator Validation

Approach	Operator validation	Functional correctness
Inputs	Dummy / synthetic tensors	Real input datasets
Measures	Throughput on isolated ops	End-to-end accuracy + throughput + latency
Pipeline	Op-level checks	Full inference pipeline (pre → infer → post)

PAVS implements the **end-to-end working pipeline** and ensures the model produces **expected outputs from real input datasets** — not just numbers from dummy inputs.

3. Developer-Friendly Inference Tools & LLM Infra

Profilers + LLM serving stacks — with tutorials for ease of adoption:

Category	Tools
Profilers	Lemonade, NPU AI Analyzer, ROCProfiler
LLM Infra	vLLM (high-throughput serving), Llama.cpp (edge inference), FastFlowLM (LLMs on NPU)

Bridging the **open-source ecosystem** ↔ **AMD HW ecosystem** — making these tools easy to use across the AMD stack.

4. Vertical Application Platform

A platform that **evolves with the client** — solving, fixing, and growing together in targeted domains:

Domain	Focus
Healthcare	Medical imaging, diagnostics
Automotive	Perception, safety systems
Industrial	Inspection, anomaly detection

Contributions(contd.) — AIG Model → Task-Specific

5. Operationalizing AIG Models for Real Applications

From Generic to Task-Specific

Took generic AIG Hub models and extended them with operation enablement, hyperparameter tuning, and backend exploration to reach application-level performance:

Backend	Role
PyTorch	Fallback path — handles missing/broken operators
torch.compile	Additional optimization on top of PyTorch
ONNX	Stable cross-device flow (CPU ↔ GPU ↔ NPU)
MIGraphX	2× improvement over torch+ROCm

MobileSAM · CenterPoint · YOLOv12 · YOLO26 — integrated into AARTS robotics pipelines, consumed by the robotics team today.

SmolVLA: Tuned for the Task

Latency	48 ms	30 ms
ROCm version	—	7.2.4
Customized to task	✗	✓

37% latency reduction through algorithmic tuning alone — on ROCm 7.2.4, without the edge-optimized kernels in ROCm 7.13 that give 2× speedup on other models.

The gains here are purely from task-specific model and algorithm optimization — not from a newer ROCm stack.

Headroom remains: upgrading to ROCm 7.13 on top of these algorithmic gains is expected to push latency even lower.

Contributions(contd.) — Ecosystem & Co-Engineering

6. Closing the Loop: Generic Model → Application Pipeline

The Full Journey

Continuing from the previous point — we don't stop at model operationalization:

1. Pick generic model from AIG Hub
2. Make it **task-specific** (tune, operationalize)
3. **Compute accuracy** on real inputs — surface gaps
4. **Communicate gaps back to AIG** for model improvement
5. Wire model into **inference pipelines** — connecting to real applications

Taking the generic model all the way to working applications — not just benchmarks.

Co-Engineering Opportunities

PAVS × Audio SDK

- Started discussions to explore co-engineering across SDK verticals
- Shared infrastructure, tooling, and pipeline patterns

Foundation Robotics — First Lighthouse Customer

- Working toward Foundation Robotics as the first co-engineering lighthouse engagement
- End-to-end story: AIG model → PAVS task-tuning → accuracy validation → AARTS inference pipeline → real robot application

Contributions(contd.) — Feedback Loop Back to AIG

Gaps Found, Communicated, and Tracked

Performance & Deprecation Risks

- **MIGraphX + ONNX is 2× faster than torch.compile** — communicated to AIG and cautioned against deprecating MIGraphX until hipDNN reaches equivalent performance
- **Concat layer broken in YOLOv26** — layer not decomposing correctly, producing garbage outputs; Jiras opened to track and get AIG attention
- **MIGraphX breaks MobileSAM** — missing operator causing failures; AIG informed to investigate the unsupported op

Operator Gaps & Tooling

- **ScatterND not available** in MIGraphX EP or ONNX EP — communicated to AIG for prioritization~(centerpoint)
- **NPU debug tools received** — now able to analyze layer-level outputs that are not producing correct results and provide richer, more precise feedback to AIG
- **Unified NPU + GPU dependency request** — GPU stack requires NumPy <2, NPU stack requires NumPy 2+; the version conflict forces separate virtual environments today. Communicated to AIG to resolve this so NPU and GPU can share a single application-level environment

PAVS acts as the functional correctness signal for AIG — we find what breaks on real inputs and real pipelines, and feed that back so the stack improves.

Contributions(contd.) — PAI SDK Platform & Edge Enablement

Infrastructure: Installer, Edge HW, and LLM Stack

Installation & Edge Hardware

- **Single installer/uninstaller** for ROCm + RyzenAI in PAI SDK — one command to set up or tear down the full stack
- Extended and maintained ROCm + RAI support for **edge HW** — required targeted patches for hardware-specific gaps
- **Yocto & LTS support** — enabling model deployment on embedded edge devices

LLM Infrastructure on AMD Stack

- Integrated **stable vLLM** — high-throughput serving with quantized LLM variants on iGPU
- Integrating **FLM framework** — running LLMs on NPU
- Bridging the open-source LLM ecosystem onto the AMD edge stack

Full-stack edge enablement: installer → hardware patches → quantized LLM serving → NPU inference → Yocto deployment.

Pain Points & Solution

Current Challenges on Edge Devices — and How We've Addressed Them

Challenges

ROCm Installation Stability

- ROCm install on edge devices is **not yet stable**
- Driver/firmware mismatches on some hardware revisions

Reboot During Installation

- ROCm installation requires a **system reboot** mid-process
- Breaks unattended / CI-driven installs

Sudo / Root Access Required

- Both ROCm and RyzenAI installers need **sudo privileges**

Solution: Dockerized PAI SDK

We have **dockerized the full setup** — ROCm + RyzenAI pre-installed, pre-configured, and ready to use out of the box:

```
# Pull and run — no installs, no reboot, no sudo
docker run physical-ai-sdk

# Start benchmarking immediately
make benchmark-all-devices metrics
```

- No local ROCm install required
- No reboot, no sudo on host
- Reproducible environment across machines
- **Ready to deliver to users today**

Docker sidesteps all three pain points — users get the full PAI SDK on the fly, without touching the host system.

SW Stack (For Reference)

